

OPTIMISATION D'UN SERVEUR OLLAMA LOCAL

sur mini-PC AMD Ryzen avec iGPU Radeon (RDNA3)

Guide pratique et to-do list — ADRASEC 77

Contexte d'usage ADRASEC : Mise en service d'un serveur LLM local (Ollama) sur mini-PC pour assister la rédaction de scénarios d'exercice, de RETEX, de documents FNRASEC, et l'analyse de logs ou de traces radio. Le présent document décrit la procédure d'optimisation permettant de passer d'une exécution CPU pure (lente) à une exécution GPU via Vulkan, sans achat de matériel supplémentaire.

1. Configuration de référence

Le retour d'expérience ci-dessous a été réalisé sur la configuration suivante. Les principes restent applicables à tout mini-PC à base d'APU AMD Ryzen 7000 ou 8000 (séries HS / U / H) équipé d'un iGPU Radeon RDNA3 (680M, 760M, 780M, 880M, 890M).

Élément	Détail
Machine de test	Geekom A7 Max (mini-PC)
Processeur	AMD Ryzen 9 7940HS (8 cœurs / 16 threads, Zen 4)
iGPU	AMD Radeon 780M (RDNA 3, 12 CU)
Mémoire vive	32 Go DDR5-4800 (2x16 Go, dual channel)
Système	Windows 11
Ollama	Version 0.22.1 (support Vulkan expérimental intégré)
Modèle LLM	mistral-nemo:12b (Q4_K_M, ~8 Go) — contexte 32k tokens
Usage cible	Serveur LAN exposé sur 0.0.0.0:11434 pour clients ADRASEC

2. Problème constaté

À l'installation par défaut sous Windows, Ollama ne détecte pas correctement les iGPU AMD Radeon de la famille 780M (architecture gfx1103). Sans intervention, le serveur tourne en mode 100 % CPU, ce qui présente plusieurs inconvénients pour un usage opérationnel ADRASEC :

- Vitesse de génération réduite (≈ 8 tokens/s sur Mistral Nemo 12B), ce qui rend la réponse longue à apparaître pour l'utilisateur en bout de chaîne.
- Saturation des 8 cœurs du processeur pendant l'inférence, empêchant l'utilisation simultanée d'autres logiciels (TCQ, VARA HF, logiciels cartographiques, etc.).
- Consommation électrique et dégagement thermique inutilement élevés sur un mini-PC en service continu.
- iGPU Radeon 780M totalement inutilisé alors qu'il est physiquement présent et capable.

Cette situation n'est pas signalée par Ollama dans son interface : la commande `ollama ps` affiche simplement « 100% CPU » dans la colonne PROCESSOR, sans avertissement ni recommandation.

3. Diagnostic — vérifications préalables

Avant toute optimisation, effectuer les contrôles suivants pour caractériser l'état initial et mesurer le gain a posteriori.

3.1. Vérifier l'état actuel d'Ollama

Ouvrir un terminal PowerShell et lancer :

```
ollama ps
```

Si la colonne PROCESSOR affiche 100% CPU, l'iGPU n'est pas exploité. C'est le cas typique sur mini-PC AMD Ryzen sous Windows en configuration par défaut.

3.2. Mesurer la vitesse de référence (avant optimisation)

Lancer une génération en mode verbose pour obtenir les statistiques détaillées :

```
ollama run mistral-nemo:12b --verbose  
>>> Écris un texte de 300 mots sur la propagation des ondes HF en bande 40 mètres
```

Noter les deux valeurs clés affichées à la fin de la génération :

- eval rate : vitesse de génération en tokens par seconde (typiquement 7 à 9 t/s en CPU pur sur Ryzen 9 7940HS avec Mistral Nemo 12B).
- prompt eval rate : vitesse de traitement du prompt d'entrée (typiquement 20 à 30 t/s en CPU pur).

3.3. Vérifier la version d'Ollama

```
ollama --version
```

La version 0.22 ou supérieure est requise pour disposer du backend Vulkan expérimental. Si la version installée est antérieure, télécharger la dernière version stable depuis ollama.com/download/windows et réinstaller.

3.4. Consulter les logs Ollama au démarrage

```
Get-Content "$env:LOCALAPPDATA\Ollama\server.log" -Tail 100 | Select-String  
"compute|library|GPU|Vulkan|VRAM"
```

Sur une configuration non optimisée, les logs contiennent typiquement les lignes suivantes :

```
OLLAMA_VULKAN:false  
experimental Vulkan support disabled. To enable, set OLLAMA_VULKAN=1  
inference compute id=cpu library=cpu  
total_vram="0 B"
```

Ces lignes confirment que Vulkan est désactivé et qu'aucun GPU n'est exploité. Elles indiquent également la solution à appliquer.

4. Procédure d'optimisation

Quatre étapes successives. Le temps total est d'environ 10 minutes, sans manipulation matérielle, sans modification du BIOS, et sans achat.

4.1. Activer le backend Vulkan dans Ollama

Vulkan est le backend graphique cross-vendor moderne qui permet à Ollama d'exploiter un iGPU Radeon RDNA3 sans nécessiter ROCm (qui n'est pas livré dans l'installateur Windows officiel d'Ollama).

Méthode recommandée : ajout de la variable d'environnement via l'interface Windows.

1. Appuyer sur la touche Windows, taper « variables d'environnement », ouvrir « Modifier les variables d'environnement pour votre compte ».
2. Dans la section « Variables utilisateur », cliquer sur « Nouvelle... ».
3. Saisir : Nom = OLLAMA_VULKAN, Valeur = 1, puis valider.
4. Cliquer sur OK pour fermer toutes les fenêtres.

Méthode alternative : via PowerShell (équivalente).

```
[System.Environment]::SetEnvironmentVariable('OLLAMA_VULKAN', '1', 'User')
```

4.2. Supprimer les variables ROCm inutiles

Plusieurs tutoriels en ligne recommandent de définir la variable HSA_OVERRIDE_GFX_VERSION pour faire fonctionner ROCm sur le 780M. Cette astuce ne fonctionne pas sur Windows car la version officielle d'Ollama n'inclut pas le runtime ROCm (uniquement disponible sur Linux ou dans le fork ollama-for-amd). Cette variable doit être supprimée si elle existe : elle est sans effet utile et ajoute du bruit dans les logs.

1. Dans la fenêtre des variables d'environnement, rechercher HSA_OVERRIDE_GFX_VERSION.
2. Si elle existe, la sélectionner et cliquer sur « Supprimer ».

4.3. Désactiver Flash Attention sur Vulkan AMD

Étape critique pour les serveurs traitant de longs prompts (usage RAG, instructions système ADRASEC étendues, historique de conversation). L'implémentation Flash Attention de llama.cpp utilisée par Ollama est mal optimisée pour Vulkan sur GPU AMD RDNA3 : son activation peut diviser par 6 la vitesse de traitement du prompt d'entrée.

1. Dans les Variables système (partie basse de la fenêtre), rechercher OLLAMA_FLASH_ATTENTION.
2. Si elle est à 1 (ou true), la modifier à 0 (ou false).
3. Si elle n'existe pas, la créer avec valeur 0.

4.4. Redémarrer Ollama complètement

Ollama lit les variables d'environnement au démarrage du service, pas en cours d'exécution. Un redémarrage propre est obligatoire.

1. Clic droit sur l'icône Ollama dans la zone de notification (flèche ^ si masquée), puis « Quit Ollama ».
2. Vérifier qu'aucun processus ne subsiste :

```
Get-Process -Name "*ollama*" -ErrorAction SilentlyContinue
```

3. Si nécessaire, forcer l'arrêt :

```
Get-Process -Name "*ollama*" -ErrorAction SilentlyContinue | Stop-Process -Force
```

4. Relancer Ollama depuis le menu Démarrer (taper « Ollama »), attendre 10 secondes que le service soit prêt.

5. Vérification du fonctionnement

5.1. Contrôle des logs après redémarrage

```
Get-Content "$env:LOCALAPPDATA\Ollama\server.log" -Tail 60 | Select-String
"OLLAMA_VULKAN|Vulkan|library|VRAM"
```

Les lignes attendues sont les suivantes :

```
OLLAMA_VULKAN:true
OLLAMA_FLASH_ATTENTION:false
inference compute id=... library=Vulkan name=Vulkan0
description="AMD Radeon 780M Graphics" type=iGPU
total="17.8 GiB" available="17.2 GiB"
```

La ligne mentionnant explicitement « AMD Radeon 780M Graphics » avec library=Vulkan confirme la détection de l'iGPU. La valeur « total VRAM » correspond à la somme de la VRAM dédiée et de la mémoire partagée allouée dynamiquement par Vulkan (typiquement 17 à 18 Go sur une machine 32 Go).

5.2. Contrôle de la répartition CPU/GPU

Charger le modèle et interroger Ollama :

```
ollama run mistral-nemo:12b
>>> bonjour
```

Puis dans un autre terminal :

```
ollama ps
```

La colonne PROCESSOR doit maintenant afficher 100% GPU (et non plus 100% CPU). La colonne SIZE indique la taille totale chargée en VRAM Vulkan (modèle + KV cache contexte).

5.3. Mesure de performance après optimisation

Refaire le test de référence avec exactement le même prompt qu'à l'étape 3.2 et noter les nouvelles valeurs.

6. Résultats mesurés

Tableau comparatif des trois configurations testées (Mistral Nemo 12B Q4_K_M, contexte 32k, KV cache Q8, prompt de référence court) :

Configuration	CPU pur (initial)	Vulkan + Flash Att. ON	Vulkan + Flash Att. OFF
eval rate (génération)	8,31 t/s	10,16 t/s	10,02 t/s
prompt eval rate (entrée)	28,68 t/s	20,23 t/s	121,41 t/s
Charge CPU pendant inférence	100 % (saturé)	Faible	Faible
Utilisation iGPU	0 %	Active	Active

6.1. Lecture des résultats

Gain en génération (eval rate) : modeste mais réel, de 8,3 à 10,0 t/s soit environ +20 %. Le Radeon 780M étant un iGPU mobile partageant la bande passante DDR5 avec le CPU (≈ 77 Go/s en dual channel), ses performances brutes sont limitées. C'est le plafond pratique sur cette gamme de matériel.

Gain spectaculaire en traitement de prompt (prompt eval rate) : facteur 6 entre Flash Attention ON et OFF (de 20 à 121 t/s). C'est le résultat qui change concrètement la perception utilisateur : un long prompt système ADRASEC (instructions + RAG + historique, typiquement 3000 tokens) passe d'environ 150 secondes d'attente avant réponse à environ 25 secondes.

Libération du processeur : bénéfice majeur souvent sous-estimé. Le CPU n'étant plus saturé, l'opérateur peut continuer à utiliser TCQ, VARA HF, des logiciels cartographiques ou tout autre outil ADRASEC pendant que le serveur LLM répond aux requêtes des autres postes du réseau local.

6.2. Synthèse opérationnelle

Pour un prompt typique de 3000 tokens en entrée et 500 tokens en sortie (cas d'usage RAG ADRASEC avec contexte documentaire), la latence totale passe de 165 secondes en CPU pur à 75 secondes après optimisation, soit un gain de 55 %.

7. Faut-il passer de 32 à 64 Go de DDR5 ?

Question fréquente lors de l'équipement de mini-PC pour usage LLM local. La réponse pour un usage ADRASEC standard (Mistral Nemo 12B ou équivalent en serveur LAN) est négative.

7.1. Pourquoi 32 Go suffisent

- Mistral Nemo 12B en Q4_K_M occupe environ 8 Go ; avec un KV cache de 32k tokens en Q8, l'empreinte totale est d'environ 11 Go.
- Avec Vulkan activé, l'iGPU dispose de 17 à 18 Go de VRAM exploitable (dédiée + partagée), soit une marge de 6 Go.
- La vitesse d'inférence est limitée par la bande passante mémoire (DDR5-4800 dual channel), pas par la capacité. Doubler la quantité de RAM ne double pas la bande passante.
- Le système Windows et les applications usuelles (TCQ, VARA, logiciels cartographiques, navigateur) consomment 4 à 6 Go ; il reste donc largement assez de RAM système.

7.2. Cas où 64 Go deviendraient utiles

- Faire tourner un modèle 70B (par exemple Llama 3.3 70B Instruct) en Q4 : ~42 Go nécessaires, mais la vitesse sera de 2 à 3 t/s sur iGPU 780M (utilisable en mode rédaction asynchrone, frustrant en mode chat interactif).
- Charger plusieurs modèles simultanément avec OLLAMA_MAX_LOADED_MODELS=3 (un modèle généraliste + un modèle de code + un modèle d'embedding pour RAG).
- Constituer et conserver en mémoire un index vectoriel volumineux sur le corpus documentaire ADRASEC (procédures ORSEC, ICS-213, RETEX, plans SATER, formations FNRASEC).

Recommandation : rester en 32 Go pour un usage standard, envisager 64 Go uniquement en cas de besoin spécifique caractérisé (multi-modèles ou RAG documentaire lourd).

8. To-do list récapitulative

Récapitulatif synthétique des actions à effectuer dans l'ordre, pour les opérateurs souhaitant reproduire l'optimisation sur leur propre matériel.

8.1. Prérequis

1. Mini-PC ou portable équipé d'un APU AMD Ryzen 7000/8000 avec iGPU Radeon RDNA3 (680M, 760M, 780M, 880M, 890M).
2. Windows 10 ou 11 à jour.
3. Drivers AMD Adrenalin Edition à jour (téléchargement depuis amd.com).
4. Ollama version 0.22 ou supérieure installé.
5. Au moins un modèle LLM téléchargé (recommandé : mistral-nemo:12b pour le français).

8.2. Diagnostic initial

1. Lancer ollama ps et vérifier la colonne PROCESSOR (constater le 100% CPU si applicable).
2. Lancer ollama run <modèle> --verbose puis une question de référence et noter eval rate et prompt eval rate.
3. Consulter les logs Ollama pour confirmer OLLAMA_VULKAN:false.

8.3. Configuration

1. Ajouter la variable utilisateur OLLAMA_VULKAN = 1.
2. Supprimer la variable HSA_OVERRIDE_GFX_VERSION si elle existe.
3. Ajouter ou modifier la variable système OLLAMA_FLASH_ATTENTION = 0.
4. (Optionnel mais recommandé) Variables système complémentaires utiles pour un serveur ADRASEC LAN :
 - OLLAMA_HOST = http://0.0.0.0:11434 (exposition sur le LAN)
 - OLLAMA_KEEP_ALIVE = 30m (maintien du modèle en mémoire 30 min)
 - OLLAMA_MAX_LOADED_MODELS = 2 (deux modèles simultanés possibles)
 - OLLAMA_KV_CACHE_TYPE = q8_0 (KV cache quantifié, économise la VRAM)
 - OLLAMA_CONTEXT_LENGTH = 32768 (contexte étendu pour RAG long)

8.4. Redémarrage et vérification

1. Quitter Ollama proprement (clic droit icône système → Quit).
2. Forcer l'arrêt des processus résiduels via PowerShell si nécessaire.
3. Relancer Ollama depuis le menu Démarrer.
4. Contrôler dans les logs la présence de library=Vulkan et name="AMD Radeon ...".
5. Relancer ollama ps et confirmer 100% GPU.
6. Refaire le test de performance et comparer aux valeurs initiales.

8.5. Pistes complémentaires (à explorer ultérieurement)

- Mise à jour BIOS du mini-PC pour activer l'option UMA Frame Buffer Size si elle est masquée par défaut (permet d'allouer 8 ou 16 Go de VRAM dédiée à l'iGPU).
- Test du fork ollama-for-amd (github.com/likelovewant/ollama-for-amd) qui inclut ROCm pour Windows et peut donner +30 à +50 % de tokens/s sur 780M.
- Évaluation de LM Studio (lmstudio.ai) comme alternative à Ollama, avec interface graphique et serveur API compatible OpenAI.
- Test de modèles alternatifs pour le français technique : Qwen 2.5 14B, Mistral Small 3 (24B), Gemma 2 27B.

9. Conseils pour le déploiement en ADRASEC

9.1. Sécurité réseau

- Limiter l'exposition d'Ollama au réseau ADRASEC interne uniquement (ne pas exposer sur Internet sans authentification : Ollama n'a pas d'authentification native).
- Utiliser un reverse proxy (Caddy, Nginx) avec authentification basique en cas d'accès distant nécessaire.
- Surveiller les logs d'accès dans server.log.

9.2. Choix du modèle pour usage ADRASEC

- mistral-nemo:12b : excellent français, contexte 128k natif, bien adapté à la rédaction de scénarios et RETEX.
- qwen2.5:14b : très bon raisonnement structuré, utile pour analyser des plans ORSEC ou des procédures.
- qwen2.5-coder:14b : aide à l'écriture de scripts Python (par exemple traitement de logs APRS, parseurs CoT TAK, automatisation TNC).
- nomic-embed-text : modèle léger d'embedding pour construire un index RAG sur la documentation ADRASEC/FNRASEC.

9.3. Bonnes pratiques d'utilisation

- Toujours vérifier le contenu généré par le LLM avant intégration dans un document officiel ADRASEC : les modèles peuvent halluciner des références réglementaires, des fréquences, ou des indicatifs.

- Conserver la mention « contenu généré par assistant IA, relu et validé par l'opérateur » dans les documents internes pour traçabilité.
 - Ne jamais utiliser le LLM pour produire du contenu opérationnel en temps réel pendant une mission active sans relecture humaine.
 - Pour les documents d'exercice, conserver les mentions obligatoires « EXERCICE - EXERCICE - EXERCICE » et « FICTIF - FICTIF - FICTIF » conformément au protocole FNRASEC.
-

Document rédigé en retour d'expérience par F1GBD

ADRASEC 77 — Seine-et-Marne / FNRASEC

Document libre de diffusion au sein du réseau ADRASEC / FNRASEC